Convenient Algebraic Programming with Coercive Subtyping

Henry Blanchette[1*] and Aaron Stump[2*]

[1]University of Maryland.
[2]Boston College.

*Corresponding author(s). E-mail(s): blancheh@umd.edu;
stumpaa@bc.edu;

Abstract

This paper presents a system for more convenient algebraic programming, a form of total functional programming where datatypes are defined as least fixed points of covariant signature functors, and recursive functions are defined as algebras. This approach requires the use of certain combinators to roll and unroll fixed points, and to convert algebras to functions which can be applied to recurse over data. These combinators are a burden for programming in this style. To alleviate that burden, in this paper we propose to insert them automatically, using coercive subtyping. The type checker generates subtyping constraints, which are then solved with the help of a custom logic programming engined called Chronolog. The subtyping axioms are given to Chronolog as rules, which uses them to solve the subtyping constraints. To avoid infinite applications of rules for unrolling signature functors, Chronolog provides a mechanism for suspending and resuming certain forms of goals. The subtyping axioms presented to Chronolog are an algorithmic version of the usual declarative rules for algebraic programming. We prove equivalence of the declarative and algorithmic rules, in Agda. We present numerous examples of algebraic programs written in our system.

Keywords: strong functional programming, type-based termination, Mendler-style recursion

1 Introduction

Strong functional programming is a form of pure functional programming where all functions are statically required to terminate on all inputs. One way to achieve this is

through an algebraic approach: datatypes are least fixed points μF of signature functors F, and recursive functions are least fixed points $(\!|a|\!)$ of algebras a. The functions are called catamorphisms. The signature functor can be seen as describing one layer of the data. The least fixed point then describes data consisting of any finite number of layers. We may unroll μF to $F\ (\mu F)$, and roll it back again. Such transformations are needed to allow construction of data by applying constructors of the signature functor, and destruction by pattern-matching against those constructors. Algebras interpret the operations declared for one layer of the data, and catamorphisms then apply algebras across all the layers. There is no other mechanism for recursion or looping in the language. All functions written in this style are guaranteed to terminate.

One hindrance to algebraic programming is the need to apply functions for rolling and unrolling signature functors, and to apply $(\!|-|\!)$ to obtain a recursive function from an algebra.

2 Motivating example

3 Syntax

$$
\begin{array}{rrcl}
\text{(Type variables)} & Y & \in & \mathscr{Y} \\
\text{(Term variables)} & x & \in & \mathscr{X} \\
\text{(Recursion type constants)} & R & \in & \mathscr{R} \\
\text{(Datatypes)} & \langle D, \langle k, \langle \overline{A,} \rangle \rangle \rangle & \in & \mathscr{D} \\
\text{(Contexts)} & \Gamma & ::= & \varnothing \mid \Gamma, (x : A) \mid \Gamma, (R : *) \\
\text{(Types)} & A, B & ::= & \iota \mid Y \mid \mu D \mid D(A) \mid A \to B \mid A \Rightarrow B \\
\text{(Terms)} & a, b, f & ::= & x \mid D.k(\overline{a,}) \mid \lambda(x : A).b \mid (f\ a) \mid \omega f(x).b \mid \\
 & & & \gamma a.\{\overline{k(\overline{x}) \mapsto b \mid}\} \mid \text{let } x : A = a \text{ in } b \mid \text{coerce } c\ a \\
\text{(Coercions)} & c, c_1, c_2 & ::= & \text{cata } D \mid \text{roll } D\ c \mid \text{unroll } D\ c \mid \text{cov } D\ c \mid \text{idData } D \mid \\
 & & & \text{idOmega } R \mid \text{subArr } c_1\ c_2 \mid \text{subAlg } D\ c
\end{array}
$$

4 Constraint-Generating Typing

- $\mathscr{D}$ is the set of datatype definitions, which are pairs of $\langle D, \mathscr{V} \rangle$ where D is an identifier and $\mathscr{V}$ is a set of constructor signatures of the form $\langle k, \langle \overline{A,} \rangle \rangle$ where k is the constructor name and each A is a constructor parameter type. The constructor

parameter types can refer to D itself, but the meta-function $\mathsf{freshenFunctorParam}(A)$ substitutes those appearances of D with a fresh type variable.

The judgment for inferring subtyping constraints is $\Gamma \vdash a : A \mid \mathcal{A} \implies \mathcal{C}$, where $\mathcal{A}, \mathcal{C}$ are sets of constraints of the form $A <:: B$. This judgment states that in context Γ, the term a is inferred to have type A under the constraint that the assumptions $\mathcal{A}$ imply the requirements $\mathcal{C}$.

$$\textsc{InferVar}\quad \frac{(x : A) \in \Gamma}{\Gamma \vdash x : A \mid \varnothing \implies \varnothing}$$

$$\textsc{InferCon}\quad \frac{\begin{array}{c} Y \text{ is a fresh type variable} \\ \langle D, \mathscr{V} \rangle \in \mathscr{D} \quad \langle k, \langle \overline{A'_i,} \rangle \rangle \in \mathscr{V} \\ (\forall i)\ A''_i := A'_i[\iota \mapsto Y] \\ (\forall i)\ \Gamma \vdash a_i : A_i \mid \mathcal{A}_{a_i} \implies \mathcal{C}_{a_i} \end{array}}{\Gamma \vdash D.k(\overline{a}) : D(Y) \mid \bigcup_i \mathcal{A}_{a_i} \implies \bigcup_i (\mathcal{C}_{a_i} \cup \{A_i <:: A''_i\})}$$

$$\textsc{InferLam}\quad \frac{\Gamma, (x : A) \vdash b : B \mid \mathcal{A}_b \implies \mathcal{C}_b}{\Gamma \vdash \lambda(x : A).b : A \to B \mid \mathcal{A}_b \implies \mathcal{C}_b}$$

$$\textsc{InferApp}\quad \frac{\begin{array}{c} A', B \text{ are fresh type variables} \\ \Gamma \vdash f : F \mid \mathcal{A}_f \implies \mathcal{C}_f \quad \Gamma \vdash a : A \mid \mathcal{A}_a \implies \mathcal{C}_a \end{array}}{\Gamma \vdash (f\ a) : B \mid \mathcal{A}_f \cup \mathcal{A}_a \implies \mathcal{C}_f \cup \{F <:: A' \to B\} \cup \mathcal{C}_a \cup \{A <:: A'\}}$$

$$\textsc{InferGamma}\quad \frac{\begin{array}{c} B, Y_i \text{ are fresh type variables} \\ \langle D, \mathscr{V} \rangle \in \mathscr{D} \\ (\forall i)\ \langle k_i, \langle \overline{A_{ij},} \rangle \rangle \in \mathscr{V} \\ (\forall i)\ A'_{ij} := A_{ij}[\iota \mapsto Y_i] \\ \Gamma \vdash a : A \mid \mathcal{A}_a \implies \mathcal{C}_a \\ (\forall i)\ \Gamma, \overline{x_{ij} : A_{ij},} \vdash b_i : B_i \mid \mathcal{A}_{b_i} \implies \mathcal{C}_{b_i} \end{array}}{\Gamma \vdash \gamma a.\{\overline{k_i(\overline{x_{ij},}) \mapsto b_i\ |}\} : B \mid \mathcal{A}_a \cup \bigcup_i \mathcal{A}_{b_i} \implies \mathcal{C}_a \cup \{A <:: \mu D\} \cup \bigcup_i (\mathcal{C}_{b_i} \cup \{B_i <:: B\})}$$

$$\textsc{InferOmega}\quad \frac{\begin{array}{c} R \text{ is a fresh type constant} \\ \Gamma, (R : *), (f : R \to A), (x : D(R)) \vdash b : B \mid \mathcal{A}_b \implies \mathcal{C}_b \end{array}}{\Gamma \vdash \omega f(x).b : D \Rightarrow B \mid \mathcal{A}_b \cup \{R <:: R\} \implies \mathcal{C}_b}$$

$$\textsc{InferLet}\quad \frac{\Gamma \vdash a : A \mid \mathcal{A}_a \implies \mathcal{C}_a \quad \Gamma, (x : A') \vdash b : B \mid \mathcal{A}_b \implies \mathcal{C}_b}{\Gamma \vdash \text{let } x : A' = a \text{ in } b : B \mid \mathcal{A}_a \cup \mathcal{A}_b \implies \mathcal{C}_a \cup \{A <:: A'\} \cup \mathcal{C}_a}$$

$$\textsc{InferCoerce}\quad \frac{c : A' \to B \quad \Gamma \vdash a : A \mid \mathcal{A}_a \implies \mathcal{C}_a}{\Gamma \vdash \text{coerce } c\ a : B \mid \mathcal{A}_a \implies \mathcal{C}_a \cup \{A <:: A'\}}$$

Fig. 1 Subtype constraint generation rules.

[TODO] Rules for inferring coercion types, $c : A \to B$

[TODO] Helper relation for inferring types of constructors of datatype, $k : (\overline{A_i}) \to D(Y)$, where Y is a fresh type metavariable.

5 Subtyping

5.1 Declarative

$$\textsc{ReflData}\ \frac{\langle D, \_\rangle \in \mathscr{D}}{\mu D <: \mu D} \qquad \textsc{Arr}\ \frac{A' <: A \quad B <: B'}{A \to B <: A' \to B'} \qquad \textsc{Cov}\ \frac{A <: A'}{D(A) <: D(A')} \qquad \textsc{Alg}\ \frac{A <: A'}{D \Rightarrow A <: D \Rightarrow A'}$$

$$\textsc{Roll}\ \frac{}{D(\mu D) <: \mu D} \qquad \textsc{Unroll}\ \frac{}{\mu D <: D(\mu D)} \qquad \textsc{Cata}\ \frac{}{D \Rightarrow A <: D \to A} \qquad \textsc{Trans}\ \frac{A <: A' \quad A' < A''}{A <: A''}$$

Fig. 2 Declarative subtyping rules.

Figure 2 shows the declarative subtyping rules. We have reflexivity rules ReflData and ReflR for datatypes and for the abstract types R introduced in the body of our Mendler algebras. Arr is the usual subtyping rule for function types, which is contravariant in the domain and covariant in the codomain. Cov expresses that the signature functor is indeed a (covariant) functor. Alg expresses that algebra types $D \Rightarrow A$ are covariant in their carriers A. Roll and Unroll together express that μF is equivalent to $F\ (\mu F)$. Cata expresses that an algebra can be coerced to a function, namely the algebra's catamorphism. Finally, there is the Trans rule, completing the definition of subtyping as a partial order.

Theorem 1 (Reflexivity) *For all types A, $A <: A$.*

5.2 Algorithmic

$$\textsc{ReflData}\ \frac{\langle D, \_ \rangle \in \mathscr{D}}{D <:: D} \qquad \textsc{Arr}\ \frac{A' <:: A \quad B <:: B'}{A \to B <:: A' \to B'} \qquad \textsc{Cov}\ \frac{A <:: A'}{D(A) <:: D(A')}$$

$$\textsc{Alg}\ \frac{A <:: A'}{D \Rightarrow A <:: \mu D \Rightarrow A'} \qquad \textsc{Roll1}\ \frac{A <:: \mu D}{D(A) <:: D} \qquad \textsc{Roll2}\ \frac{\mu D <:: A}{\mu D <:: D(A)} \qquad \textsc{Cata}\ \frac{B <:: D \quad A <:: C}{D \Rightarrow A <:: B \to C}$$

Fig. 3 Algorithmic subtyping rules.

Figure 3 presents the algorithmic subtyping rules. We use similar names as for the declarative rules, but distinguish the algorithmic judgment by writing $A <:: B$ (where the declarative judgment is written $A <: B$). Critically, the algorithmically problematic Trans rule is not present. A consequence of this is that now several rules that did not have premises in the declarative formulation, here do. For example, compare the declarative (on the left) and algorithmic Cata rules:

$$\frac{}{D \Rightarrow A <: D \to A} \qquad \frac{B <:: D \quad A <:: C}{D \Rightarrow A <:: B \to C}$$

The algorithmic rule needs to take into account subtyping relationships between the components in the conclusion, because transitivity is not available to take those into account outside a use of this rule. We see a similar change from the declarative (Roll and Unroll) to the algorithmic (Roll1 and Roll2) rules for rolling and unrolling the signature functor.

Theorem 2 (Reflexivity) *For all types A, $A <:: A$.*

Lemma 3 (Decrease) *For all rules of Figure 3, the sizes of the constraints in the premises are less than the size of the constraint in the conclusion.*

Theorem 4 (Decidability) *It is decidable whether or not two types are in the algorithmic subtyping relation.*

Proof Bottom-up application of the rules to ground types is guaranteed to terminate, by Lemma 3. Therefore, proof search may be used as a decision procedure for algorithmic subtyping of ground types. □

5.3 Equivalence of declarative and algorithmic subtyping

5.4 Formalization in Agda

6 Implementation with Chronolog

7 Returning to the Examples

8 Related Work

Turner introduced the term *strong functional programming*, presented several arguments in favor of the approach, and proposed an *elementary* way to realize such a language, in the form of static checks for structural recursion [1]. Mendler proposed a recursor using an abstract type to ensure that recursive calls are permitted only on subdata for an inductive type [2].

9 Conclusion and future work

References

[1] Turner, D.A.: Elementary Strong Functional Programming. In: Proceedings of the First International Symposium on Functional Programming Languages in Education. FPLE '95, pp. 1–13. Springer, Berlin, Heidelberg (1995)

[2] Mendler, N.P.: Inductive types and type constraints in the second-order lambda calculus. Annals of Pure and Applied Logic **51**(1), 159–172 (1991)